

NOTE: This project was developed using Visual Studio 2026. Peer reviewers using VS Code can run the project with dotnet build and dotnet run, and then open Swagger manually at <https://localhost:5001/swagger/index.html>. Please see detailed instructions in the README.pdf file in this project's GitHub repository.

UserManagementAPI Project Summary

Phase 1 – User Management API Design and Development

Phase 1 focused on developing the foundation of the User Management API. The project was implemented as a RESTful API using ASP.NET Core and followed conventional routing through `/api/Users`. An in-memory data store was selected for this phase to simplify development and allow the primary focus to remain on API functionality and behavior.

The API was designed according to standard HTTP conventions. GET requests were used to retrieve information, POST requests were used to create new users, PUT requests were used to update existing users, and DELETE requests were used to remove users. A strongly typed `User` model provided a consistent structure for user data.

Swagger/OpenAPI was enabled to provide interactive API documentation and testing. Particular attention was also given to maintaining a clean controller design and producing readable, maintainable code.

Features Implemented

The completed API supported all four CRUD operations. Users could retrieve all records through `GET /api/Users` or retrieve an individual user through `GET /api/Users/{id}`. New users could be created through `POST /api/Users`, with IDs assigned automatically. Existing records could be modified through `PUT /api/Users/{id}`, while `DELETE /api/Users/{id}` removed users from the system.

Basic error handling was also incorporated. Requests for users who did not exist generated a 404 Not Found response, while invalid JSON generated a 400 Bad Request response. Swagger UI automatically documented the available endpoints, and Postman was used to verify API behavior using realistic HTTP requests.

Testing Results

Testing with Swagger and Postman confirmed that all CRUD operations functioned as expected. A GET request for all users successfully returned the default users, Alice and Bob, with a 200 OK response. Retrieving user ID 1 returned Alice's record and also produced a 200 OK response.

A POST request successfully created a new user, Charlie Davis, and returned 201 Created. A subsequent PUT request changed Charlie to Charles Davis and returned 204 No Content. A GET request confirmed that the changes had persisted.

Finally, a DELETE request successfully removed user ID 3 and returned 204 No Content. A subsequent GET request verified that the user no longer existed.

Problems Encountered and Resolutions

Several problems occurred during development and testing. Initially, Postman could not connect to the API because the application was not running in Visual Studio. The problem was resolved by ensuring that Visual Studio was running the application and that the console displayed the appropriate "Now listening" message.

A POST request later returned 404 Not Found because of an incorrect endpoint or missing JSON request body. Correcting the endpoint and configuring the request body as raw JSON resolved the problem.

During PUT testing, a 400 Bad Request response occurred because additional text had accidentally been pasted after the JSON body. Removing the non-JSON text corrected the request.

A DELETE request also produced 404 Not Found because the requested user ID did not exist. Since the in-memory user list reset whenever the application restarted, the user first had to be recreated through a POST request before the DELETE operation could be tested.

Phase 1 Outcome

Phase 1 successfully implemented all required CRUD endpoints and demonstrated that the API behaved according to REST conventions. Testing with both Swagger and Postman was successful. The debugging process also provided practical experience diagnosing API connectivity, request formatting, endpoint, and data-persistence issues.

Phase 2 – Debugging the API with Copilot

Overview

Phase 2 expanded the User Management API beyond basic CRUD functionality by introducing stronger validation, improved error handling, edge-case testing, and additional security measures.

The goal was to create a REST service that behaved consistently when processing both valid and invalid requests. All endpoints were extensively tested through Swagger, including normal operations and unusual or incorrect inputs. Several security and robustness measures were also implemented beyond the original course requirements.

CRUD Operations and Model Validation

The API continued to provide the complete set of CRUD endpoints for retrieving, creating, updating, and deleting users.

Model validation was significantly strengthened. The `fullName`, `email`, and `department` fields were made mandatory, and minimum-length validation was introduced for the user's full name. Email addresses were checked using the `[EmailAddress]` validation attribute. The `required` keyword was also added to model properties to provide stricter binding behavior.

Together, these changes helped prevent malformed, incomplete, or otherwise invalid JSON payloads from being accepted by the API.

Error Handling

Error handling was standardized throughout the application. Invalid payloads and type mismatches produced `400 Bad Request`, while requests involving missing or invalid user IDs produced `404 Not Found`. Successful deletions returned `204 No Content`.

Validation errors were returned using RFC 9110-compliant `application/problem+json` responses. Error messages were designed to be clear and understandable, providing information such as when a user could not be found, an email address was invalid, or a full name did not meet the minimum length requirement.

Security Improvements

Additional response headers were used to strengthen the application's security posture. These included Content Security Policy protections, `X-Content-Type-Options`, `X-Frame-Options`, `X-XSS-Protection`, and `Referrer-Policy`.

Although these measures were not specifically required by the course, they were examined as part of the effort to improve the security and robustness of the API.

Edge-Case Testing

Extensive edge-case testing was conducted using Swagger UI. POST requests were tested with missing fields, invalid email addresses, incorrect data types, and empty request bodies. Each invalid scenario correctly produced a `400 Bad Request`.

PUT requests were tested using nonexistent IDs, empty request bodies, invalid data, mismatched IDs, and incorrect data types. Invalid requests generated the appropriate `400` or `404` responses. Testing also established the application's behavior when the ID in the URL differed from the ID supplied in the request body.

GET-by-ID requests were tested with negative IDs, zero, and nonexistent IDs. Each correctly produced `404 Not Found`.

DELETE requests were similarly tested using nonexistent and negative IDs. A double-delete scenario was also examined. The first deletion successfully returned `204 No Content`, while attempting to delete the same user a second time correctly returned `404 Not Found`.

Finally, all users were deleted to test the GET-all endpoint with an empty data set. The API correctly returned an empty JSON array (`[]`) with a `200 OK` response rather than returning null or a `404` response.

Additional Enhancements

Several improvements went beyond the basic course requirements. Security protections were examined through response headers, and error responses were checked to ensure that sensitive information was not exposed.

Validation was strengthened through required properties, type-mismatch detection, and protection against invalid or incomplete JSON. REST behavior was made more consistent, including predictable deletion responses and returning an empty array when no users existed.

Developer usability was also improved through clearer error messages, standardized problem-detail responses, and comprehensive Swagger documentation and testing.

Problems Encountered and Resolutions

During development, POST and PUT initially accepted some malformed input. Stronger validation was added through `[Required]`, `[MinLength]`, `[EmailAddress]`, and the `required` keyword.

Swagger occasionally displayed "Undocumented" for 204 No Content responses. Testing confirmed that the API behavior itself was correct, while additional XML documentation could be introduced later if desired.

Testing also revealed potential confusion when the URL ID and body ID differed. The implemented behavior was verified so that the URL ID controlled the resource being updated.

Finally, the GET-all endpoint was tested after removing every user. It was configured to return an empty array consistently, eliminating ambiguity about how an empty data set should be represented.

Phase 2 Outcome

All endpoints successfully passed normal-operation and edge-case testing. By the end of Phase 2, the User Management API provided substantially stronger validation, predictable error handling, and improved security.

The result was a more robust REST service capable of responding consistently to both valid and invalid requests.

Phase 3 – Implementing and Managing Middleware

Design and Architecture

Phase 3 extended the project by introducing authentication, persistent storage, middleware, and a more structured application architecture.

JWT tokens were selected for secure authentication, while password hashing and verification were implemented according to secure development practices. The project adopted a layered architecture that separated controllers, services, and Entity Framework Core responsibilities. Standardized responses were also used to provide consistent behavior for successful and unsuccessful operations.

Authentication

An `AuthController` was implemented with two principal endpoints: `POST /api/Auth/register` for user registration and `POST /api/Auth/login` for authentication.

JWT token generation included the issuer, audience, expiration time, and signing key. Passwords were hashed rather than stored directly, and Entity Framework Core was introduced to provide persistent user storage.

Global exception-handling middleware was implemented to generate consistent error responses throughout the application. The authentication endpoints were also integrated with Swagger so that they could be documented and tested interactively.

Authentication Testing

Registration testing confirmed that successful registration returned a JWT together with the user object. Attempts to register duplicate usernames were rejected, and requests containing missing required fields generated validation errors.

Login testing confirmed that correct credentials generated a JWT. Incorrect credentials were rejected, as were attempts to authenticate users who did not exist.

Database testing verified that the Users table had been successfully created and populated. Passwords were confirmed to be stored as hashes rather than as plain text. Swagger testing also confirmed that all authentication endpoints were functioning and returning the expected JSON responses.

Problems Encountered and Resolutions

Several important problems were encountered during Phase 3. A missing dependency-injection registration caused an "Unable to resolve service for IAuthService" error. Restoring the appropriate registrations in `Program.cs` corrected the problem.

Entity Framework Core errors also occurred because of a mismatch in the database schema. The SQLite database was recreated, and the migrations were reapplied to bring the schema back into alignment with the application.

Password verification initially failed because of an issue in the hashing and verification logic. Correcting that logic resolved the authentication problem.

Swagger also sent empty request bodies during some tests. Ensuring that the JSON was entered into the editable request-body field corrected the issue. Finally, generic error responses were improved by enhancing the application's global exception-handling middleware.

Product Catalog API

Within the project, the Users API functions as the project's "product catalog" by managing user entities. It provides complete CRUD functionality through `/api/Users`, uses Entity Framework Core with SQLite for persistent storage, and integrates Swagger for documentation and testing.

The API supports retrieving all users, retrieving an individual user by ID, creating users, updating existing users, and deleting users.

API Design Decisions

A layered architecture separates controllers, services, and Entity Framework Core. Data Transfer Objects (DTOs) provide a clean separation between API models and database entities, while dependency injection is used throughout the service layer.

REST principles guide the API design. Resources use clear, resource-based routes, and GET, POST, PUT, and DELETE retain their conventional HTTP meanings. JSON responses and HTTP status codes are handled consistently.

Security is provided through JWT authentication for protected endpoints, password hashing before database storage, and validation of incoming requests.

Maintainability is improved through centralized exception handling, modular service interfaces, and a clear separation of concerns between application components.

Middleware Pipeline Configuration

Exception Handling

The exception-handling middleware captures unhandled exceptions and converts them into standardized JSON error responses. This approach also helps prevent internal stack traces and other implementation details from being unintentionally exposed to API clients.

Routing and Endpoints

Routing is configured through the ASP.NET Core middleware pipeline, with controller attribute routing defining the individual API endpoints. This provides a clear and predictable relationship between HTTP requests and controller actions.

Authentication and Authorization

JWT validation middleware controls access to protected resources. Tokens are validated by checking properties such as the issuer, audience, signing key, and expiration time before authenticated access is permitted.

CORS

Cross-Origin Resource Sharing (CORS) is configured to support Swagger and the local development tools used during development and testing.

Framework Middleware

The project also uses standard ASP.NET Core framework functionality, including HTTPS redirection, content negotiation, JSON model binding, and JSON response formatting.

Overall Project Outcome

Across the three phases, the User Management API progressed from a basic in-memory CRUD application to a substantially more complete REST service.

Phase 1 established the fundamental CRUD operations and REST conventions. Phase 2 strengthened the application through validation, error handling, security considerations, and comprehensive edge-case testing. Phase 3 introduced authentication, password hashing, persistent SQLite storage through Entity Framework Core, dependency injection, and centralized middleware.

The resulting project demonstrates the progression from basic API development toward a structured, secure, maintainable, and testable ASP.NET Core application.